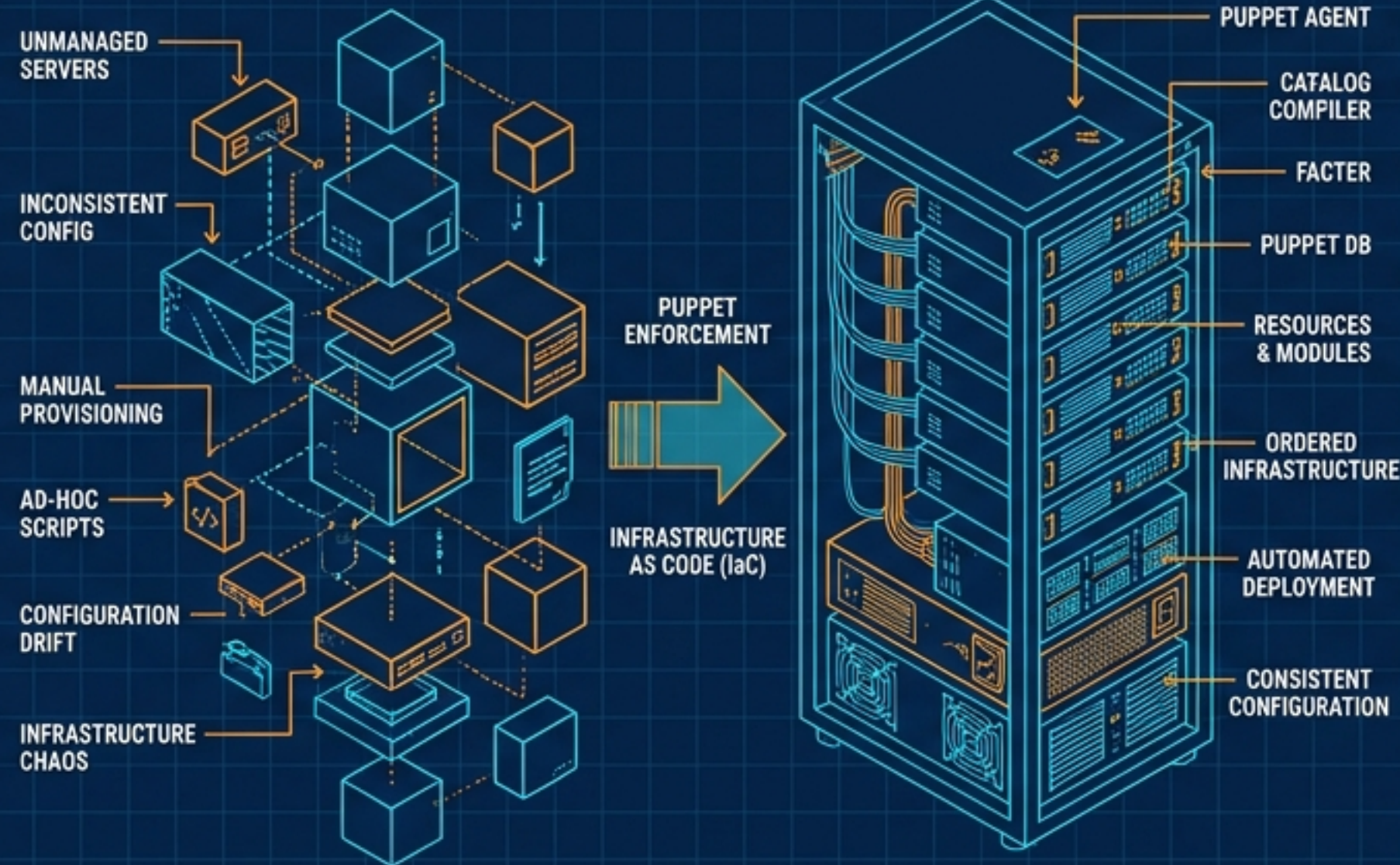
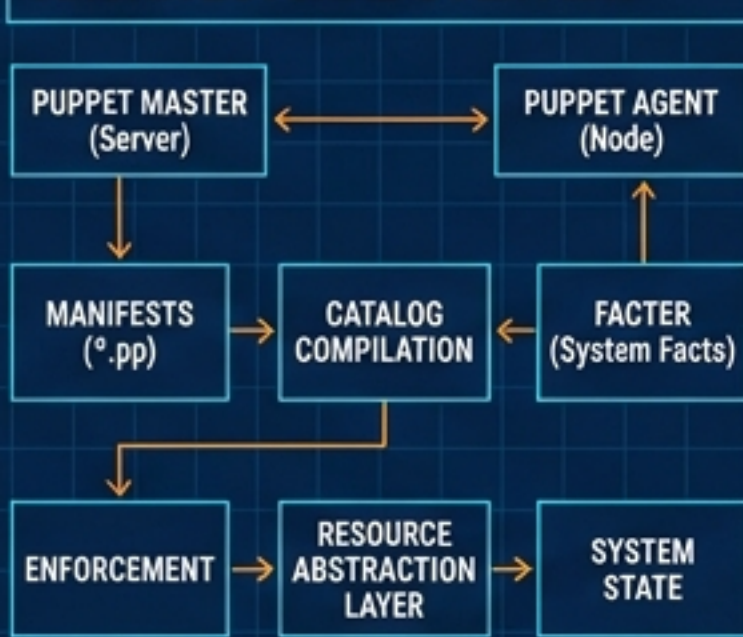


Puppet Fundamentals

Infrastructure as Code: From Architecture to Manifests



ARCHITECTURAL OVERVIEW



MANIFEST DEVELOPMENT

```
[RESOURCE TYPE] : [TITLE] {  
  [ATTRIBUTE] => [VALUE],  
}
```

EXAMPLE: BASIC MANIFEST (*.pp)

```
file { '/etc/motd':  
  ensure => file,  
  content => "Welcome to the Puppet Lab\n",  
  mode => '0644',  
} ->  
package { 'nginx':  
  ensure => installed,  
}
```

TARGET & CONTEXT

TARGET AUDIENCE	4th-Year CSE Students
LEARNING OUTCOME	Explain Puppet architecture & develop basic manifests
LAB ENVIRONMENT	Local execution via `puppet apply`

FIGURE 01: INFRASTRUCTURE ORDER THROUGH PUPPET AUTOMATION

The Paradigm Shift: Desired State Management

Imperative (Bash)

```
if ! dpkg -l | grep -q nginx; then  
  apt-get install nginx;  
fi
```

Focuses on **HOW** to execute the steps.

Declarative (Puppet)

```
package { 'nginx':  
  ensure => present,  
}
```

Defines **WHAT** the final state must be.



Idempotency: This code can safely run 1,000 times. If the system **matches** the **desired state**, Puppet does **nothing**. If it **drifts**, Puppet **fixes** it.

Enterprise Architecture vs. Standalone Lab

Production: Agent-Server Model

Puppet Server

Facts sent up
(HTTPS/SSL)

Agent Node 1

Agent Node 2

JSON Catalog
sent down

Agent Node 3

Today's Lab: Standalone Execution



Code and compilation happen **locally** without a network server. Perfect for **development, testing, and today's hands-on lab.**

The Core Components of Puppet

	Manifests	Your Puppet code files (<code>.pp</code>). The blueprints defining the desired state.
	Facter	The system inventory tool. Discovers OS facts like IP, OS family, and CPU.
	Catalog	A compiled JSON document. Translates your code into a node-specific to-do list.
	Puppet Agent	The application running on the target node that enforces the Catalog.
	Puppet Server	The primary node that stores code and compiles Catalogs for Agents.

Preparing Your Local Environment

macOS (Native Execution)

```
# Install via Homebrew
$ brew install puppet

# Verify Installation
$ puppet --version
```

Windows (WSL2 Ubuntu Execution)

```
# Update & Install via Apt
$ sudo apt update && sudo apt install puppet

# Verify Installation
$ puppet --version
```

Note: We are using Puppet Core for this lab. No virtual machines or enterprise servers are required for local puppet apply execution.

Anatomy of a Puppet Manifest

Title
(Must be unique
per type)

Resource Type
(The system entity)

```
user { 'developer':  
  ensure      => present,  
  managehome => true,  
}
```

Attributes
(Properties of
the resource)

Values

Styling Rules:

File extension: `.pp`

Two-space indents
(no tabs)

Arrows (`=>`) aligned
for readability

Trailing commas
required

Foundational Resources & State

Package

```
package { 'git':  
  ensure => present,  
}
```

File

```
file { '/tmp/hello.txt':  
  ensure => file,  
  content => 'Hello World',  
}
```

Service

```
service { 'ssh':  
  ensure => running,  
  enable => true,  
}
```



Dependency Warning: Puppet does NOT apply resources top-to-bottom like a script. It builds a dependency graph. Explicit relationships (e.g., `require => Package['git']`) must be defined if order matters.

Organizing Code: Classes and Modules

```
my_module/  
├── manifests/  
│   └── init.pp  <- Defines main class  
├── files/       <- Static config files  
└── templates/  <- Dynamic templates
```

Classes

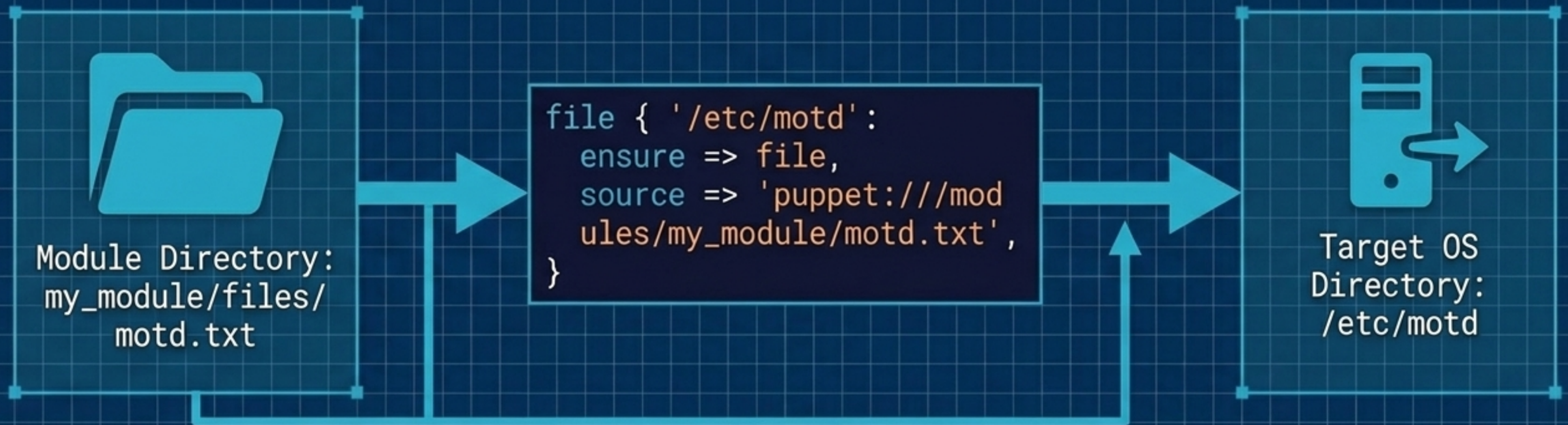
Named blocks of Puppet code that bundle related resources together for logical grouping.
(Note: These are singletons, not Object-Oriented classes).

Modules

Portable, shareable directory structures containing classes, static files, and dynamic templates.

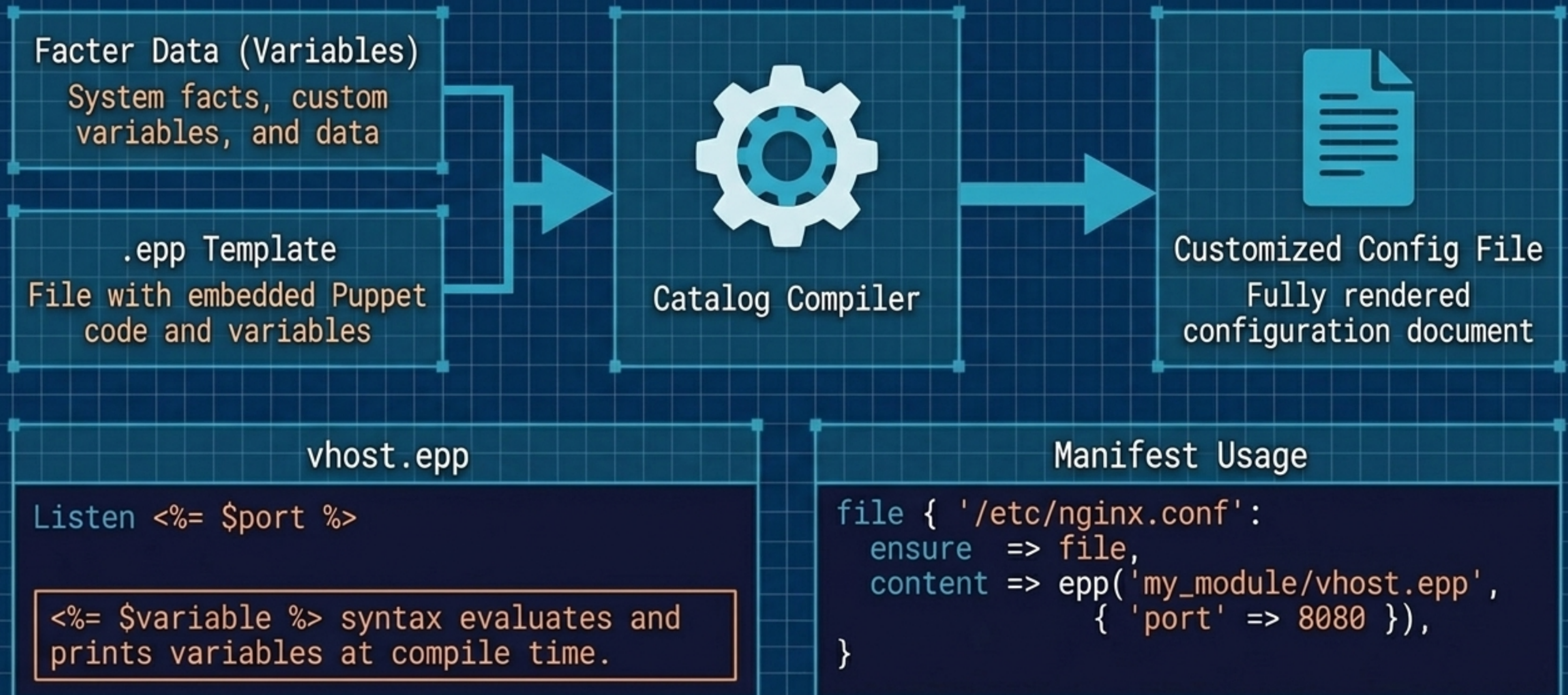
Modules are the fundamental building blocks of scalable infrastructure.

Distributing Static Content (File Server)

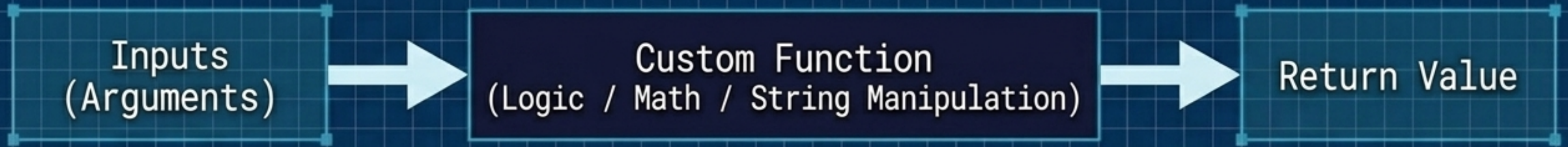


The Concept: Drop pre-written static configuration files onto a system without hardcoding every line in the manifest. The `'puppet:///'` URI automatically maps to the hidden `'files/'` directory inside the module.

Dynamic Configurations with EPP Templates



Extending Logic: Custom Functions



```
# Define the function in my_module/manifests/my_func.pp
function my_module::greet(String $name) >> String {
    return "Hello, ${name}!"
}
```

```
# Execute inside a manifest
$message = my_module::greet('CSE Students')
```

Modern Puppet functions are written directly in the declarative Puppet language (no Ruby required), keeping core manifests clean and logic encapsulated.

The Abstraction Layer: Types vs. Providers

Resource Type (Cross-Platform Interface)

```
package { 'vim': ensure => present }
```



Factor OS Discovery

Is this macOS
or Ubuntu?

macOS



Provider: brew install vim

Ubuntu

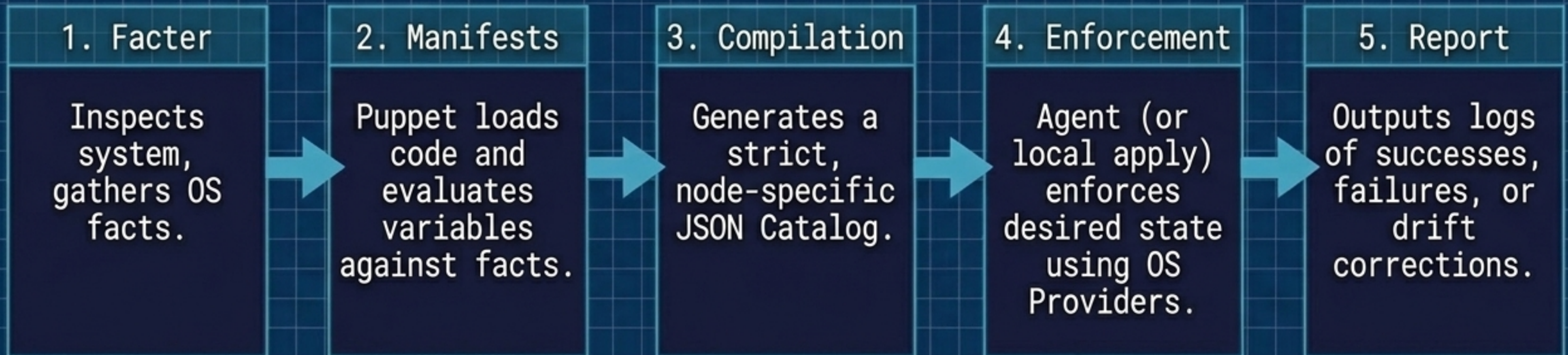


Provider: apt-get install vim

Key Takeaway: You write one manifest. Puppet automatically selects the correct provider based on Factor OS data to execute commands under the hood.

The Compilation Pipeline

Handled completely locally during
today's puppet apply lab.



Hands-on Lab: Your First Local Manifest

1. Create a file named setup.pp
2. Add a package resource (macOS: 'tree', WSL: 'htop')
3. Add a user resource with your name
4. Execute: puppet apply setup.pp
5. Execute the exact same command a second time.

```
$ puppet apply setup.pp
```

Observe Idempotency: On your second run, watch the output. Notice how nothing changes because the desired state has already been met!

Syllabus Recap: Puppet Fundamentals

Architecture

Server compiles, Agent enforces (`Lab: apply`).

Components

Manifests, Facter, Catalog, Agent, Server.

Installation

Native Mac via `Brew`, WSL2 Ubuntu via `Apt`.

Coding Style

Declarative DSL, unique titles, strict formatting.

Classes & Modules

Logical groupings and directory structures.

File Server

`puppet:///` for distributing static files.

Templates

`EPP files` for dynamic variable injection.

Custom Functions

Reusable logic blocks written in Puppet DSL.

Type Providers

Types define 'What', Providers execute 'How'.